

Core-Delta KV 선택 실험 보고서 (2026-09-04 ~ 09-05)

작성: 2026-09-05 16:00. 원데이터 results/smoke/{cd,wr,inc}*.jsonl, 판별 요약 *_summary.md. 이 문서는 하루 반 동안 다섯 번 바뀐 설계를 순서대로 기록하고, 각 설계에서 실제로 켜 수치와 그 해석, 그리고 무엇이 아직 확인되지 않았는지를 한 곳에 모은다. 용어는 처음 나올 때 풀어 쓴다.

0. 한 줄 요약

질문 없이 미리 고른 이미지 기준(KVzip 방식)을 층·KV head 단위로 적용하면 화면 KV의 5%만으로도 질문을 미리 아는 상한에 거의 붙고(0.76~0.84 vs 0.82~0.86), 여기에 과거 질문·답의 사용 기록을 4분의 1만 켜면 예산이 빠듯할 때 질문이 쌓일수록 더 좋아진다(화면 4번째 질문 +4.4%p, 사진 3번째 질문 +6.4~7.9%p, 신뢰구간 0 제외). 사용 기록만으로 고르면 무너지고, 예산이 20% 이상이면 이미지 기준만으로 충분하다.

1. 무엇을 검증하려 했나

출발점은 사용자가 9월 4일 아침에 쓴 원안(VLM_idea_v1_kvzip_core_delta.md로 보존)이다.

- 캐시(KV cache): 모델이 이미지와 질문을 읽으면서 token·층·KV head마다 만들어 두는 중간 기억. 답을 생성할 때 이 기억을 다시 읽는다. 압축 = 그중 일부만 남기는 것.
- core: 질문을 모르는 상태에서 "이미지 자체를 보존할 가치"로 고른 조각.
- delta: 질문에 맞춰 추가로 고른 조각.
- 가설: 같은 예산 안에서 core와 delta를 함께 고르면, 둘 중 하나만 쓴 것보다 낫다.

하루 동안 설계가 다섯 번 바뀌었다. 바뀐 이유는 모두 "무엇을 재는 실험인가"에 대한 사용자의 정정이었고, 각 단계의 수치가 다음 단계의 근거가 됐다.

단계	설계	압축 시점	사용자의 정정
A	시각 token만, 층·head 공통 mask, 현재 질문의 점수로 delta	질문 도착 전	원안 그대로 (§ 3: KV는 하나, mask만 결합)
B	프롬프트 전체 KV, 층·head 단위	질문을 읽은 뒤 잘라내기	"전체 KV로" → "질문 읽은 뒤 줄이는 건 의미가 없다"
C	쓰기 시점 core + 읽기 시점 작은 delta (전체 KV는 차가운 저장소)	질문 전 core, 질문 시 delta	"KVzip과 저장량이 다르니 공정하지 않다"
D	DRAM 보관분 B만 남기고 삭제, 그 안에서 core·delta	질문 전	"DRAM에 올릴 만큼만, 나머지는 버려야"
E	같은 화면에 질문이 반복될 때, 이미지 기준 + 누적 사용 기록으로 매 질문 재선택	새 질문 도착 전	"이미지 기준과 이미지+과거 질문 기준으로 고르고 싶다"

E가 최종 설계이고 가장 많은 판을 돌렸다. A와 C는 참고 수치로 남겼고, B와 D는 실행 초기에 중단했다.

2. 공통 실험 규약

- 모델: Qwen2.5-VL-7B-Instruct (주), Qwen3-VL-8B-Instruct (일부). fp16, V100.
- 데이터: ScreenQA discovery 172장(앱 화면, 화면당 질문 5~8개, 근거 상자 있음), GQA discovery 300장(일상 사진, 사진당 5~8개, scene graph로 근거 자동 라벨). 질문 쌍의 근거 겹침 라벨: T2 = 같은

근거, T3 = 다른 근거.

- 선택 단위: (층, KV head, token) 세 짝. Qwen2.5는 28층 × 4 KV head, Qwen3는 36층 × 8. 예산은 세 짝 수의 비율이며 K/V 본체 byte와 정비례한다.
- 실행 방식: 전체 KV를 만든 뒤 attention 차단표로 "남기지 않은 조각을 못 보게" 한다. 계산 결과는 실제로 지운 것과 같다. 메모리 절감은 회계값이지 실측이 아니다.
- 앞 4개 token(attention sink)은 항상 보존하고 예산에 포함한다.
- 지표: 정답 완전 일치(EM). 헤드라인은 **압축 없이도 맞힌 질문만**(FULL 정답) 집계한 값이며, 전체 질문 값도 각 요약에 있다. 신뢰구간은 이미지 단위 bootstrap 95%, 조건 간 차이는 짝지은 차이.
- § 3 검증: 이미지 뒤의 텍스트를 바꿔도 시각 위치의 K/V가 같은가. Qwen2.5는 같은 입력 반복이 정확히 0, 텍스트 교체 시 상대 오차 K 3e-4, V 4e-3(fp16 커널 잡음). Qwen3-VL은 10배 큰 차이 (V 상대 오차 1e-2~3e-2, 특정 원소가 통째로 바뀔)가 나왔고 원인은 미확인이다.

3. 결과 1 — 시각 token만, 5% 예산, 현재 질문의 점수 (설계 A)

Qwen2.5. 이미지 기준 = KVzip 재구성 점수(질문 대신 "보이는 것을 그대로 옮겨라"로 설명문을 생성시켜 설명문이 각 조각을 참고한 최대치), 질문 기준 = 현재 질문의 attention. α = 이미지 몫.

조건	화면 (FULL 정답 421)	화면 (전체 516)	사진 (FULL 정답 429)	사진 (전체 900)
무작위	0.209	0.180	0.536	0.341
이미지만 ($\alpha=1$, KVzip)	0.577	0.496	0.527	0.313
$\alpha=0.75$	0.907	0.767	0.627	0.357
$\alpha=0.5$	0.933	0.787	0.793	0.444
$\alpha=0.25$	0.941	0.791	0.860	0.473
질문만 ($\alpha=0$)	0.933	0.781	0.876	0.476

해석: 현재 질문을 아는 선택이 사실상 상한이고, 이미지 기준을 섞어도 화면에서는 동률($\pm 1\%p$, CI가 0 포함), 사진에서는 손해다. 원안 § 4.5의 "질문만보다 일관되게 높다" 조건은 성립하지 않았다. 다만 "예산의 절반을 이미지 기준에 줘도 현재 질문 정답률이 안 떨어진다"는 관찰이 설계 C의 근거가 됐다.

4. 결과 2 — 쓰기 시점 core + 읽기 시점 delta (설계 C, 참고 수치)

Qwen2.5 × 화면 169장. 이미지 접두 KV(system 문구 + 시각 1,238 token) 위에서 core는 질문 없이 KVzip 점수로, delta는 질문 도착 시 질문 attention으로 고른다. 전체 KV는 차가운 저장소에 둔다고 가정(저장량은 줄지 않음). 행 = core 크기, 열 = delta 크기, 칸 = FULL 정답 질문 EM.

core \ delta	0%	1%	2.5%	5%
0%	—	0.591	0.729	0.827
2.5%	0.724	0.922	0.948	0.967
5%	0.824	0.941	0.960	0.967
10%	0.891	0.952	0.969	0.971

- 같은 1% 전송량에서 core의 가치: +0.33 [+0.28, +0.38] (2.5% core), +0.35 (5% core).
- 무작위 core(5%) + 1% delta는 0.727 → core의 내용이 중요(+0.21).

- token 단위(모든 층·head 공통)로 1%를 고르면 0.195, head 단위는 0.591 → 빠듯한 예산에서는 head 단위가 필수.
- 이 판은 전체 KV를 보관하므로 KVzip(나머지 삭제)과 저장량 비교가 아니다. 줄이는 것은 질문 시점의 GPU 상주량과 전송량이다.

5. 결과 3 — 같은 화면에 질문이 반복될 때의 누적 선택 (설계 E, 주 결과)

절차. 전체 KV는 DRAM에 있다고 보고, 질문 1~4가 차례로 온다. 질문 t를 답하기 전에 다음 두 기준으로 활성 집합을 정한다. 이미지 기준(KVzip 재구성 점수, 화면당 1회)과 기록 기준(이전 질문 1..t-1과 그때의 답이 각 조각을 참고한 attention의 평균, 전체 캐시 위에서 측정). α 몫을 이미지 순위로 먼저, 나머지를 기록 순위로 채운다. 질문 t는 선택에 관여하지 않는다.

- keep: 질문마다 예산 B를 다시 고름(크기 유지, 구성 교체).
- grow: 질문마다 B만큼 추가(20% 단계면 20→40→60→80%, 5% 단계면 5→10→15→20%).
- 대조: 무작위(같은 크기), 상한(질문 t 자신의 attention으로 고름), 전체 캐시.
- $\alpha=1$ = 이미지만(KVzip), $\alpha=0$ = 기록만. 단계 1은 기록이 없어 모두 같다.

5.1 화면 (Qwen2.5, 170장, FULL 정답 568문항)

keep 20%:

단계	이미지만	$\alpha=0.75$	$\alpha=0.75$ - 이미지만 [95% CI]	기록만	상한	무작위
t=1	0.952	(동일)	—	—	0.946	0.680
t=2	0.907	0.921	+0.014 [0, +0.036]	0.707	0.936	0.750
t=3	0.931	0.965	+0.035 [0, +0.069]	0.688	0.951	0.729
t=4	0.934	0.934	0	0.766	0.956	0.745

grow 20→80%: 40%부터 무작위도 0.94, 60%부터 0.98 이상. 방법 차이를 볼 수 없다(천장).

keep 5%:

단계	이미지만	$\alpha=0.75$	$\alpha=0.75$ - 이미지만 [95% CI]	기록만	상한	무작위
t=1	0.762	(동일)	—	—	0.816	0.231
t=2	0.836	0.850	+0.014 [-0.021, +0.057]	0.400	0.857	0.271
t=3	0.806	0.826	+0.021 [-0.021, +0.069]	0.465	0.840	0.243
t=4	0.832	0.876	+0.044 [+0.007, +0.088]	0.409	0.781	0.190

grow 5→20%: 10% 0.907(이미지만) vs 0.886($\alpha=0.75$), 15% 0.917 vs 0.938 (+0.021, CI 0 포함), 20% 0.934 vs 0.920.

근거 겹침: 이 표본의 질문 쌍은 T3(다른 근거) 377, T2(같은 근거) 14뿐이다. T3만 봐도 $\alpha=0.75$ 가 t=2·3·4에서 +1.5 / +1.5 / +4.6%p로 같은 방향이다.

5.2 사진 (Qwen2.5, 250장, FULL 정답 587문항; T2 209 / T3 207)

keep 20%: $\alpha=0.75$ - 이미지만 = +0.027 [0, +0.060] (t=2), +0.050 [0, +0.107] (t=3), 0 (t=4). 기록만도 0.89~0.93으로 이미지만과 비슷(같은 물체를 거듭 묻기 때문). grow는 40%부터 포화.

keep 5%:

단계	이미지만	$\alpha=0.75$	$\alpha=0.75 - \text{이미지만} [95\% \text{ CI}]$	기록만	상한	무작위
t=2	0.698	0.765	+0.067 [-0.007, +0.141]	0.698	0.913	0.557
t=3	0.714	0.750	+0.036 [-0.029, +0.100]	0.764	0.914	0.600
t=4	0.729	0.786	+0.057 [0, +0.121]	0.707	0.929	0.621

grow 5→20%:

단계	크기	이미지만	$\alpha=0.75$	기록만	$\alpha=0.75 - \text{이미지만}$	기록만 - 이미지만
t=2	10%	0.839	0.886	0.872	+0.047 [-0.007, +0.101]	+0.034
t=3	15%	0.850	0.914	0.929	+0.064 [+0.014, +0.121]	+0.079 [+0.021, +0.136]
t=4	20%	0.914	0.936	0.936	+0.021	+0.021

근거 겹침(keep 5%, t=2): 같은 근거(T2)에서 이미지만 0.688, 기록만 0.958, $\alpha=0.75$ 0.771. 다른 근거(T3)에서 이미지만 0.717, 기록만 0.565, $\alpha=0.75$ 0.772.

5.3 화면, Qwen3-VL (172장 중 144장, FULL 정답 487문항; 부분 판, 20% 단계만)

keep 20%: 이미지만 0.829 / 0.777 / 0.776 / 0.864 (t=1..4). $\alpha=0.75$: 0.835 (+0.058 [0, +0.116]), 0.832 (+0.056 [-0.008, +0.120]), 0.864 (0). $\alpha=0.5$: 0.843, 0.856 (**+0.080 [+0.016, +0.144]**), 0.881. 기록만 0.694 / 0.758 / 0.788. 상한 0.901 / 0.935 / 0.924. 무작위 0.77 / 0.73 / 0.69. Qwen3는 20%에서 이미지의 정답률이 낮아(0.78) 여지가 크고, 기록을 섞은 이득이 Qwen2.5보다 크다. grow는 40%부터 포화.

5.4 전체 질문 기준 (헤드라인과 방향이 같은지 확인)

FULL 정답 질문만이 아니라 전체 질문으로 봐도 부호와 순서는 같다. 예: 화면 keep 5% t=4 이미지만 0.692 → $\alpha=0.75$ 0.715 (+0.023 [-0.012, +0.058]); 사진 keep 5% t=3 0.393 → 0.440 (+0.047 [+0.010, +0.087]), t=4 0.407 → 0.447 (+0.040 [+0.007, +0.077]).

6. 종합 해석

- 이미지 기준(KVzip 방식)을 층·head 단위로 옮긴 것만으로 큰 진전이다.** D6(token 공통 mask)의 화면 5% 0.54가 head 단위 + sink 보호로 0.76~0.84가 됐고, 질문을 미리 아는 상한(0.82~0.86)과의 차이가 0.02~0.06으로 줄었다. 이는 "질문 없이 미리 줄여도 손해가 거의 없다"는 결과이며, KVzip이 텍스트에서 주장한 것을 화면에서 재현한 셈이다.
- 기록 기준은 조건부로 유효하다.** 예산이 빠듯하고(5%) 질문이 쌓일수록 이미지 75% + 기록 25% 결합이 이미지만보다 좋아지며, 화면(근거가 매번 다름)에서도 4번째 질문에서 +4.4%p, 사진(근거 반복)에서는 3번째 질문에서 +6~8%p다. 근거가 반복되는 질문에서는 기록의 이득이 +27%p까지 간다.
- 기록만으로는 안 된다.** 이미지 기준 없이 기록만 쓰면 화면에서 0.40까지 떨어진다. 이미지 기준이 맞이고 기록은 보정이다. 원안의 "core는 붕괴를 막는 닻"이라는 직관이 맞았다.
- 예산이 넉넉하면 이득이 없다.** 20% 이상에서는 이미지만으로 상한 근처이고 grow 방식은 무작위조차 0.94 이상이라 방법이 구분되지 않는다. 주장은 5~10% 구간으로 제한해야 한다.
- 현재 질문을 알 때의 delta는 별개의 큰 효과다.** 결과 2에서 core 5% + 현재 질문 delta 1%가 0.94였다. 이는 질문 시점에 작은 조각을 가져오는 두 저장소 구조의 근거이며, 결과 3의 "과거 질문 기록"과는 다른 주장이다.

7. 한계와 주의

- 모든 정답률은 차단표 시뮬레이션이고, 메모리·지연은 회계값이다. head 단위 물리 삭제는 별도 세션 엔진(`core/ragged_kv.py`, 다른 세션이 구현, 화면 1장 smoke만 수행)에서 가능성이 확인됐지만 이 보고서의 수치는 그 엔진으로 잦 것이 아니다.
- 기록 기준 점수는 전체 캐시 위에서 잦 정확한 attention이다. 실제 시스템에서 압축된 캐시만 보고 답할 때의 점수와는 다를 수 있다(세션 엔진은 후자를 잦다).
- 화면 표본의 T2 쌍이 14개뿐이라 "같은 근거 반복" 조건은 사진에서만 검정됐다.
- Qwen3 화면 판은 144/172장에서 중단됐고, Qwen3 사진 판과 5% 단계는 돌리지 않았다.
- 9월 4일 저녁 GPU 3이 PCIe 버스에서 사라졌고(재부팅 필요), 그 오류 로그($78\text{GB} \times 2$)가 디스크를 가득 채워 실험이 한 차례 중단됐다. 로그를 비우고 rsyslog 규칙으로 억제한 뒤 GPU 0·2로 재개했다. GPU 0은 83도에서 throttling해 GPU 2의 절반 속도였다.
- 설계 B(질문 읽은 뒤 잘라내기)와 설계 D(DRAM 보관분 삭제)는 코드만 있고 결과가 없다.

8. 구현 자산

파일	역할
<code>vlm_diagnosis/core/core_delta.py</code>	고정 예산 결합 <code>core_delta_keep</code> , §3 검증
<code>vlm_diagnosis/core/kv_select.py</code>	(층, head, token) 세 짝 점수·선택, 층별 차단표 주입(<code>PerHeadEviction</code>), 회계
<code>vlm_diagnosis/exps/core_delta_sweep.py</code>	설계 A 러너
<code>vlm_diagnosis/exps/core_delta_full_kv.py</code>	설계 B 러너 (다른 세션이 image-only 신호로 개정)
<code>vlm_diagnosis/exps/core_delta_write_read.py</code> , <code>core_delta_dram.py</code>	설계 C·D 러너
<code>vlm_diagnosis/exps/core_delta_reuse.py</code>	과거 질문 1개 기준 재사용 판 (12장에서 중단)
<code>vlm_diagnosis/exps/core_delta_incremental.py</code>	설계 E 러너 (<code>keep/grow</code> $\times \alpha$, 대조군, 역순 처리 옵션)
<code>vlm_diagnosis/scripts/core_delta_*_analysis.py</code>	판정 표(bootstrap CI, 층별)
<code>vlm_diagnosis/scripts/chain_core_delta_*.sh</code>	GPU 제한 순차 실행
<code>tests/test_core_delta.py</code> , <code>tests/test_kv_select.py</code>	선택 규칙·차단표 단위 테스트
<code>docs/RECURRENT-SESSION.md</code> 및 <code>core/session_*.py</code> , <code>core/ragged_kv.py</code> , <code>core/pair_session.py</code>	다른 세션이 만든 물리 삭제 세션 엔진 (미평가)

요약 파일: `results/smoke/cd_q25_{sqa,gqa}_summary.md`, `wr_q25_sqa_summary.md`, `inc_q25_{sqa,gqa}[_b5]_summary.md`, `inc_q3_sqa_partial_summary.md`.

9. 다음 단계 (제안)

1. 화면 표본에서 "같은 근거 질문을 앞에 둔 순서"와 "다른 근거만 앞에 둔 순서"를 짝 라벨(T2 122쌍, 45장)로 재구성해, 기록 기준이 언제 도움이 되는지를 화면 도메인에서 통제 실험으로 확정한다.
2. Qwen3 사진 판과 5% 단계를 마저 돌려 모델 불변성을 채운다.
3. 세션 엔진에 사전 점수(KVzip 재구성)와 결합 규칙(고정 분할) 옵션을 열어, 이 보고서의 20% 조건 하나를 물리 삭제로 재현하고 실측 메모리·지연을 붙인다.
4. Visual Dialog(10턴 대화, 예/아니오 45%)로 대화 형태의 규모 검증을 추가한다.

5. Qwen3-VL의 § 3 이상(텍스트 교체 시 V 원소 급변)을 `scripts/kv_invariance_probe.py`로 진단한다.